

Bounded LLM Adaptation in a Multi-Agent Simulation of the High-End GPU Compute Market

Junrong Yi[†]

Dian Jin[†]

Haotian Chen[†]

Fudan University, Shanghai, China

[†]Equal contribution.

Abstract

We present a synthetic multi-agent market for high-end GPU computing services and use it to compare several ways of placing a large language model inside a repeated pricing and capacity decision loop. Three provider archetypes compete for training, enterprise-SLA, and spot customers under noisy demand, constrained supply, customer reallocation, inter-provider transfer, backlog, and reputation feedback. Five policies share the same environment: a deterministic heuristic, direct LLM action generation, context-augmented direct control, bounded adaptive strategy updates, and context plus bounded adaptation. The four LLM policies complete 864 agent-round decisions across six scenarios without heuristic fallback. In the reported single-seed experiment, context plus adaptation has the highest profit in five of six scenarios and the lowest aggregate mean final backlog. Under the supply shock, it records profit 4327, fulfillment 96.08%, and final backlog 3.65, compared with backlog around 11 for the other LLM policies. The experiment is a mechanism study rather than a forecast of real GPU prices or profitability: it uses a synthetic demand process, a 12-round horizon, and one random seed.

Keywords: LLM agents; multi-agent simulation; GPU computing; repeated market interaction; bounded adaptation; agent-based economics

1 Introduction

High-end GPU computing is a useful setting for studying repeated agent decisions because capacity is scarce, demand is bursty, customers differ in service requirements, and shortfalls can carry over through backlog. The project builds a synthetic market rather than fitting real transaction data. Its purpose is to test agent architectures under a common environment, not to estimate actual provider prices or market shares.

The design separates the market from the decision policy. Demand generation, customer allocation, shortage handling, transfer, settlement, and reputation are deterministic environment modules once the random seed and scenario are fixed. The LLM is placed inside one of several policy wrappers. This distinction matters: observed behavior is produced jointly by the environment, provider priors, safety constraints, and the language model.

LLM-based agents have been used to generate persistent behavior in interactive simulations; the Generative Agents architecture is a prominent example of combining language models with memory and reflection.[1] Our experiment asks a narrower systems question: in a market simulator, should the LLM directly choose actions, receive more context, or update a bounded strategy state that is translated into actions by conventional code?

2 Market and Methods

2.1 Synthetic Market

The simulator combines three heterogeneous providers, three customer segments, a stochastic demand process, segment-wise allocation, customer reallocation, inter-provider transfer, backlog, and reputation feedback. These market mechanisms are shared by every policy configuration, so the policy comparison changes the decision module while leaving settlement and capacity accounting fixed.

2.2 Provider archetypes

The repository implements three providers with deliberately different operating profiles. Their economic roles are not only narrative labels: the configuration fixes distinct forecasting, pricing, allocation, and risk styles together with different cost, capacity, inventory, and reputation parameters. Hyperscaler is scale-oriented, PremiumCloud is SLA-oriented, and SpotBroker is designed as a flexible spot-market intermediary. These priors are encoded in capacity,

Table 1: Provider parameters from the project report.

Provider	Initial reputation	Brand	Base / floor price	Max. quantity
Hyperscaler	0.65	0.05	4.60 / 4.60	120
PremiumCloud	0.85	0.30	5.40 / 4.40	120
SpotBroker	0.55	0.10	4.90 / 3.80	80

brand, price floors, and policy styles. Consequently, later role differentiation should be interpreted as consistency with the environment design rather than spontaneous discovery of real provider identities.

2.3 Customer segments and demand

Total demand is split into training-burst (45%), enterprise-SLA (35%), and spot-workload (20%) segments. Segment choice uses a softmax attraction model involving price, brand, reputation, delivery performance, and provider-role preference. Training demand is relatively price-sensitive and favors Hyperscaler; enterprise demand puts stronger weight on reputation and contract service; spot demand is more price-sensitive and favors SpotBroker.

Demand is generated from trend, seasonality, AR(1) noise, and scenario shocks. Agents observe a noisy version rather than the latent true demand. This creates a deliberate information gap between the environment and the decision makers: an agent sees a noisy current observation and a limited recent history, while the latent demand process and competitors' internal policy states remain hidden.

2.4 Segment-wise allocation and customer mobility

The repository's final environment replaces a single homogeneous demand pool with three separately allocated segments. For segment g , provider i receives a score of the form

$$S_{ig} = w_b^{(g)} B_i + w_R^{(g)} R_i + w_{sla}^{(g)} R_i^{del} + b_{ig} - w_p^{(g)} p_i, \quad (1)$$

and the segment is divided by a softmax over S_{ig} . The default weights make enterprise-SLA demand much more reputation-sensitive ($w_R = 1.45$) than spot demand (0.45), while spot demand is more price-sensitive ($w_p = 1.15$) than enterprise demand (0.35). Contract fractions are 10%, 75%, and 5% for training, enterprise, and spot demand, respectively.

Allocation is followed by a second market-clearing layer. Unserved customers can move to providers that still have surplus capacity, ranked by a generic combination of brand, reputation, and price. The baseline reallocation fill rate is 1.0. In the supply-shock scenario it is deliberately reduced to 0.60, leaving local shortages for the transfer mechanism rather than allowing customer switching to absorb the entire shock. This ordering—segment choice, customer reallocation, then provider transfer—is important when interpreting the transfer statistics.

Shortage Buffers, Payoff, and Reputation. Shortage is handled in two stages. First, customers that cannot be served may be reallocated to another provider. Second, remaining shortage can trigger inter-provider transfer. The model uses a logistic cooperation probability

$$P_{coop} = \sigma \left(\alpha_0 + \alpha_1 R_{req} - \alpha_2 \frac{y_{prov}}{A_{prov}} - \alpha_3 dump_{req} \right), \quad (2)$$

where R_{req} is requester cooperation reputation, y_{prov}/A_{prov} is provider load, and $dump_{req}$ marks low-price dumping in the previous round.

Profit includes sales, transfer income and expenditure, production, inventory holding and depreciation, SLA/backlog penalties, price-pressure cost, and repricing cost. Transfer carries a 5% markup. Reputation combines delivery, pricing, and cooperation components with weights 0.5, 0.3, and 0.2 and an exponential update rate of 0.25. A quote below marginal cost triggers a dump flag; shortage above 10% of allocated demand triggers a default flag.

These mechanisms create a repeated-game feedback loop. A provider can gain short-run volume through price or capacity choices, but service failures affect future allocation through backlog and reputation.

2.5 Backlog and reputation state updates

The implemented SLA mechanism is a lightweight queue rather than a full contract simulator. At the start of a round, previously accumulated backlog is served before current demand when capacity permits. A segment's contract fraction determines how much newly unmet demand enters the queue. Unserved old backlog becomes *late units* and incurs an

SLA queue penalty. This design gives service failures persistence: shortage in one round can consume capacity and affect profit in the next.

Reputation is also decomposed rather than updated from profit alone. Delivery, pricing, and cooperation sub-scores receive weights 0.5, 0.3, and 0.2, followed by an exponential update with rate 0.25. A quote below marginal cost is recorded as a dump event, while shortage exceeding 10% of allocated demand is recorded as a default event. These event flags affect both reporting and future cooperation. Thus reputation connects customer allocation and provider-to-provider liquidity without giving the LLM direct authority over either mechanism.

Policy Architectures. All policies share a four-stage pipeline: demand forecast, pricing, supply decision, and risk review.

Heuristic. No external model is called. Rules use recent demand and trend, provider-specific base prices, inventory, reputation, and volatility. This policy is also the fallback for LLM parsing failure.

Direct LLM. The model returns forecast demand, price, and quantity directly. The action is parsed and then clipped by the Risk Gate.

Context. Direct action generation is supplemented by a six-round compressed history and active risk signals such as service and inventory pressure.

Adaptive. The LLM does not choose the final action. Instead, it updates six bounded strategy variables: risk tolerance, price aggressiveness, demand sensitivity, inventory caution, shock responsiveness, and competitor reactivity. Deterministic policy code maps this state to actions.

Context + Adaptive. The bounded strategy update receives the compressed context as well. This is the most constrained use of the LLM in the project: environment mechanics, state bounds, risk checks, and settlement remain outside the model.

The distinction between direct action generation and bounded state updates is central to the experiment. It allows a test of whether an LLM is more useful as a low-level controller or as a slower strategic modifier.

2.6 Deterministic Risk Gate

Inspection of the repository clarifies that no LLM action is passed directly to settlement. The shared Risk Gate applies role-specific checks and then rounds price and quantity to the legal action grid. It first estimates a capture share from equal-share prior, relative reputation, relative price, and role bias, clipped to $[0.12, 0.72]$. This share and the forecast define an inventory target. A growth-tolerant Hyperscaler can add capacity after shortage; the SLA-guard policy can lift a low quote toward the market average and reduce quantity under service pressure; the inventory-guard policy caps quantity and can lift price in high volatility. A final inventory-target cap and low-reputation fallback are then applied. The resulting action is constrained to the provider’s price floor/ceiling, quantity step, and maximum quantity.

For the adaptive policies, the LLM therefore operates one level further away from settlement. It proposes deltas to six strategy coordinates, those deltas are bounded, deterministic code maps the resulting state into a draft action, and the same Risk Gate reviews the draft. This architecture is the main reason we describe the method as *bounded* LLM adaptation rather than autonomous LLM trading.

2.7 Context as compressed evidence

The context variants do not simply append the complete transcript of previous rounds. The implementation maintains a bounded rolling context (six recent rounds in the final experiment) containing selected outcomes and market signals. For adaptive control, the prompt includes the previous strategy state, the latest action and outcome, current market information, peer reputations, shock information, and the compressed history. The instruction explicitly asks for small strategy changes unless repeated shortage, backlog, or losses justify a stronger response, and explicitly forbids direct price, quantity, or forecast output. This makes the context an evidence channel for strategy updating rather than an unrestricted memory buffer.

3 Experimental Design

Six scenarios are used: baseline, price war, supply shock, high volatility, no reputation, and no transfer. They are implemented as parameter transformations of one baseline environment rather than separate simulators. In *price_war*, overall price weight rises to 1.15, reputation weight falls to 0.85, spot demand rises to 40%, transfer markup falls to 2%, and price-pressure cost is activated at rate 0.55. In *supply_shock*, a 60-unit shock occurs at round 8, AR noise rises, maximum transfer increases from 15 to 35, and reallocation fill falls to 0.60. In *high_volatility*, AR noise, observation noise, and weekly seasonal amplitude are increased. The *no_reputation* ablation zeros reputation and SLA weights in allocation and reputation aggregation, while *no_transfer* disables provider transfer entirely.

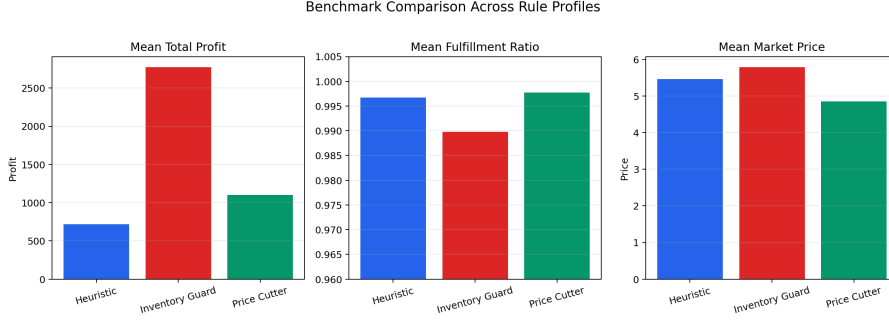


Figure 1: Benchmark comparison across the evaluated policy configurations.

Table 2: Selected scenario changes verified from the repository configuration.

Scenario	Main intervention
Baseline	Default segmented market, reallocation, transfer, backlog, reputation
Price war	40% spot share; stronger price sensitivity; pressure-cost rate 0.55
Supply shock	Round-8 shock +60; max transfer 35; reallocation fill 0.60
High volatility	Larger AR noise, observation noise, and seasonal amplitude
No reputation	Reputation/SLA weights removed from allocation and reputation aggregation
No transfer	Transfer disabled; maximum transfer set to zero

All five policies use seed 7 and 12 rounds per scenario. The four LLM modes use Gemini 3 Flash at temperature 0 with one retry. Across four modes, six scenarios, three agents, and 12 rounds, the experiment records 864 LLM decisions and no heuristic fallback. The common environment and seed make within-run comparisons controlled, but one seed is not enough for confidence intervals over stochastic demand or model behavior.

4 Results

4.1 Aggregate Results

Adding context to direct action generation changes aggregate profit little and increases defaults in this run. The adaptive modes produce substantially higher simulated profit, even though their forecast MAE is much worse. In this environment, point-forecast accuracy is therefore not aligned with the final decision objective. The bounded strategy state can support profitable inventory and pricing behavior without minimizing one-step demand error.

Context + Adaptive has the highest profit in five of six scenarios. The exception is price war, where Adaptive reaches 3597 and Context + Adaptive 3501. The strategy analysis attributes this to a more cautious response under contextual risk signals: price aggressiveness and risk tolerance fall, which protects against low-price pressure but gives up market share.

Supply-Shock Stress Test. The supply shock produces the clearest separation. Direct LLM uses the least transfer capacity and finishes with backlog 11.10. Both adaptive policies use much more transfer and maintain fulfillment above 95%. Context + Adaptive ends with backlog 3.65, well below the other three policies.

The higher profit under the shock should not be interpreted as a claim that real supply disruptions increase provider profit. The scenario raises demand but does not add all emergency procurement, expansion, financing, or long-term contractual costs. Service variables such as fulfillment, transfer, defaults, and backlog are the more informative stress indicators.

Customer Segments and Provider Roles. Under Context + Adaptive, the segment results broadly follow the encoded roles. In baseline, PremiumCloud captures a reported 96.65% enterprise signal but has fulfillment 94.66% and three defaults, reflecting the higher service pressure of enterprise demand. During price war, SpotBroker’s spot capture rises to 51.49%. Under supply shock, Hyperscaler captures 50.30% of training demand, while PremiumCloud’s enterprise capture is 74.58% but fulfillment falls to 88.88% with five defaults.

Provider profit also remains differentiated. Under Context + Adaptive, Hyperscaler has the highest cumulative profit in all six reported scenarios: 1523 in baseline, 1446 in price war, 1604 in supply shock, 1389 in high volatility, 1421 without reputation, and 1321 without transfer. SpotBroker is particularly useful as a flexible buffer; in supply shock it earns 1422 while maintaining 99.07% fulfillment and zero defaults.

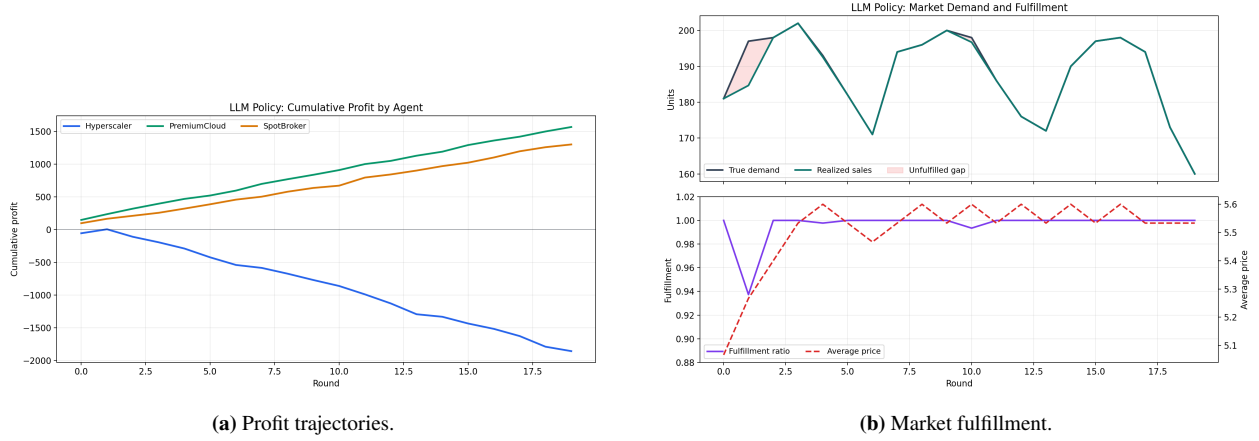


Figure 2: Round-level behavior from the repository’s LLM experiments.

Table 3: Aggregate results for the four LLM policies across six scenarios.

Policy	Profit	Fulfillment	Defaults	Mean final backlog	Forecast MAE
Direct LLM	19,770	98.17%	14	2.23	6.76
Context	19,855	97.54%	27	2.24	7.06
Adaptive	22,349	98.35%	25	3.17	34.17
Context + Adaptive	22,681	98.46%	25	1.34	34.30

The final strategy states give a more direct view of the bounded-adaptation mechanism. At the end of supply shock, Hyperscaler has high risk tolerance, price aggressiveness, and demand sensitivity with low inventory caution. PremiumCloud has higher inventory caution and lower competitor reactivity. SpotBroker has the largest shock responsiveness and competitor reactivity. Because the provider priors are already different, this result is best read as preservation and adaptation of role-specific behavior rather than emergence from identical agents.

Mechanism Ablations. The no-reputation scenario removes reputation and SLA weights from customer choice and cooperation reputation from transfer, while retaining brand, role, cost, capacity, and price-floor differences. It therefore measures the marginal role of reputation signals, not anonymous competition among homogeneous firms. Context + Adaptive records profit 3693, fulfillment 98.96%, five defaults, and backlog 2.63 in this ablation.

The no-transfer scenario removes the second shortage buffer but retains customer reallocation. Context + Adaptive records profit 3591, fulfillment 98.29%, seven defaults, and zero final backlog. The fact that fulfillment remains high indicates that reallocation is already an important first buffer; transfer matters for more localized shortages rather than all service recovery.

Environment-Agent Separation and Decision Timing. The source implementation separates market mechanics from agent policy. The environment generates demand, performs customer allocation, processes shortage recovery, settles profit, updates reputation, and writes the next-round history. The agent pipeline is responsible for demand forecasting, pricing, supply choice, and risk review. This separation is methodologically important: a policy cannot change the settlement rule or bypass capacity accounting merely because an LLM proposes an action.

Each round follows the same fixed sequence. The environment generates a noisy demand realization and exposes the permitted observation to each provider. Providers then produce forecast, price, and quantity decisions. The environment allocates the three customer segments, reallocates local shortages, invokes inter-provider transfer when needed, moves unresolved contractual demand into backlog, settles revenue and costs, updates reputation, and compresses the outcome into the next observation history.

The provider definitions also include decision-style priors beyond the scalar parameters in Table 1. Hyperscaler is momentum-oriented, share-seeking, and growth-tolerant. PremiumCloud is signal-smoothing, premium-preserving, and SLA-guarding. SpotBroker is volatility-reading, spread-seeking, and inventory-guarding. These priors are part of the environment design and therefore matter when interpreting later role differentiation.

Customer Allocation and Two-Stage Shortage Recovery. Total demand is split into training-burst, enterprise-SLA, and spot segments. The source environment assigns each segment with a softmax over provider attractiveness, where attractiveness depends on brand, reputation, role preference, and quoted price. Training demand has a price weight of

Table 4: Four LLM policies in the supply-shock scenario.

Policy	Profit	Fulfillment	Transfer	Defaults	Final backlog
Direct LLM	3597	93.22%	15.15	9	11.10
Context	3696	95.10%	50.31	9	11.19
Adaptive	4238	95.30%	92.52	7	10.92
Context + Adaptive	4327	96.08%	83.96	7	3.65

0.75 and a Hyperscaler bias; enterprise demand has reputation weight 1.45, a 75% contract share, and a PremiumCloud bias; spot demand has price weight 1.15 and a SpotBroker bias.

When a chosen provider cannot satisfy its allocation, the environment first attempts customer reallocation. Only the remaining shortage proceeds to inter-provider transfer. The analysis models the cooperation probability with a logistic rule of the form

$$P_{coop} = \sigma \left(\alpha_0 + \alpha_1 R_{req} - \alpha_2 \frac{y_{prov}}{A_{prov}} - \alpha_3 dump_{req} \right), \quad (3)$$

where requester cooperation reputation raises acceptance, provider load lowers it, and a recent low-price dumping flag can reduce willingness to cooperate. Transfer is therefore neither automatic nor purely price-based.

This ordering gives the simulator two distinct buffers. Reallocation represents customers moving to another seller before a contract failure becomes persistent. Transfer represents providers sharing capacity after reallocation is insufficient. Contractual shortage that survives both stages enters backlog, so a one-round mismatch can affect later rounds.

Payoff and Reputation Mechanics. The provider payoff is deliberately broader than sales revenue. It includes current sales and backlog delivery, transfer income and expenditure, linear and quadratic production cost, inventory holding and depreciation, SLA shortage penalties, overdue-backlog penalties, spot-market price pressure, and repricing cost. In compact notation, the analysis writes

$$\pi_{i,t} = (s_{i,t} + b_{i,t}^{del})p_{i,t} + TR_{i,t}^{in} - TR_{i,t}^{out} - C_{i,t}^{prod} \quad (4)$$

$$- h_i I_{i,t} - C_{i,t}^{dep} - \rho_i^{sla} short_{i,t} - \rho_i^{late} \kappa late_{i,t} \quad (5)$$

$$- C_{i,t}^{pressure} - \mu_i |p_{i,t} - p_{i,t-1}|. \quad (6)$$

Transfer carries a 5% markup. The backlog multiplier is reported as $\kappa = 0.75$, and the price-war scenario sets the price-pressure rate to 0.55 while baseline uses zero.

Reputation is not a single delivery score. Delivery, pricing, and cooperation components receive weights 0.5, 0.3, and 0.2, respectively, and the combined score is updated with an exponential rate of 0.25. A quote below marginal cost triggers a dumping flag; shortage exceeding 10% of allocated demand triggers a default flag. These rules explain why short-run profit maximization can have delayed consequences in later customer allocation and transfer cooperation.

Policy Distinctions. The heuristic baseline uses recent demand and trend, provider-specific base prices, inventory, reputation, and volatility. Supply is clipped by the provider’s maximum new quantity, and every action passes the Risk Gate. Direct LLM replaces the main forecast, price, and quantity choice with JSON output from the model but retains the heuristic as a parsing fallback.

Context mode adds a compressed six-round history and active signals such as service pressure and inventory pressure. Adaptive mode changes the interface more fundamentally: the model updates six bounded state variables—risk tolerance, price aggressiveness, demand sensitivity, inventory caution, shock responsiveness, and competitor reactivity—while deterministic policy code maps those states to final actions. Context + Adaptive combines the bounded update with the compressed history.

This distinction matters for interpreting the results. Direct and Context ask the model for low-level actions. Adaptive and Context + Adaptive ask it for a slower strategic state that is subsequently constrained by role priors and deterministic code. The experiment is therefore partly a comparison of LLM control interfaces, not merely a comparison of prompt lengths.

Mechanism Calibration. The strategy appendix records an earlier calibration stage. Initial scenarios produced several effects that were difficult to interpret: price-war profit was too high, removing reputation was too benign, and the supply shock behaved more like a market-wide collapse than a localized mismatch. The final environment adds price-pressure cost, removes both reputation and SLA preference weights in the reputation ablation, moderates the supply shock, and damps bounded strategy updates.

The calibration is not additional evidence that one policy is superior; it documents how the simulator was made more internally interpretable before the aligned comparison. In particular, price-war profit falls below baseline after adding

Table 5: Adaptive-policy results before and after mechanism calibration, as reported in the strategy appendix.

Scenario	Profit pre	Profit post	Fill pre	Fill post	Backlog pre	Backlog post
Baseline	3586	3716	97.2%	99.3%	0.02	0.52
Price war	3745	3597	98.4%	99.8%	0.87	0.00
Supply shock	3475	4238	79.3%	95.3%	40.96	10.92
High volatility	3462	3707	97.7%	98.9%	2.40	1.32
No reputation	3679	3533	98.5%	98.9%	1.64	3.23
No transfer	3544	3559	97.3%	97.9%	3.52	3.02

Table 6: Selected segment and service results under Context + Adaptive.

Scenario	Provider	Training	Enterprise	Spot	Fulfillment	Defaults
Baseline	Hyperscaler	45.97%	23.41%	18.03%	100.00%	0
Baseline	PremiumCloud	48.38%	96.65%	11.41%	94.66%	3
Baseline	SpotBroker	41.16%	28.65%	28.67%	99.80%	0
Price war	Hyperscaler	41.86%	13.83%	41.08%	99.21%	0
Price war	PremiumCloud	30.47%	63.32%	11.18%	97.65%	1
Price war	SpotBroker	32.83%	20.66%	51.49%	99.03%	0
Supply shock	Hyperscaler	50.30%	24.99%	19.90%	96.90%	2
Supply shock	PremiumCloud	37.90%	74.58%	9.06%	88.88%	5
Supply shock	SpotBroker	39.54%	28.82%	27.07%	99.07%	0

price pressure, and the supply shock leaves enough residual capacity for reallocation and transfer to matter instead of overwhelming every provider simultaneously.

LLM Call Audit and Reproducibility. Each of the four LLM modes generates $6 \times 12 \times 3 = 216$ agent-round decisions, for 864 decisions in total. The strategy appendix reports zero decision fallback in all four modes and zero strategy-update fallback in the two adaptive modes. This audit matters because otherwise a strong result could be produced partly by the heuristic fallback rather than by the intended LLM interface.

The analysis names the principal implementation modules: `timeseries.py` for demand, `environment.py` for allocation and settlement, `agents/pipeline.py` for the four-stage decision chain, and `agents/llm.py` for context handling. It also gives `pj-ag4-run` as a single-scenario entry point and `pj-ag4-scenarios` for scenario sweeps. The reported full experiment uses the same seed, scenario definitions, round count, allocation mechanics, settlement logic, and logging schema across policies; only the agent decision module changes.

This design supports reproducibility of the synthetic experiment, but it does not remove stochastic uncertainty. Temperature zero reduces sampling variation from the model interface, while the demand process remains stochastic under the fixed seed. Multi-seed experiments would be needed to estimate variability across demand paths.

Scenario-Level Behavior. The price-war exception is informative. Context + Adaptive has the highest profit in five of six scenarios, but Adaptive leads in price war with 3597 versus 3501. The strategy appendix attributes the difference to contextual risk signals making the combined policy more cautious under strong price sensitivity and price-pressure cost. In other words, more information is not unconditionally beneficial when the controller maps it into conservative behavior.

The supply shock provides a different stress pattern. Demand rises by 60 units, about 30%, at round 8 and reallocation efficiency is reduced. Adaptive modes respond with much larger transfer volumes than direct modes. Context + Adaptive finishes with backlog 3.65 rather than roughly 11, suggesting that persistent service-pressure context is most useful when the strategic state, rather than the raw action, is being updated.

High volatility is less decisive. The analysis explicitly notes that 12 rounds are too short to create a large separation among policies after increasing AR(1) and observation noise. This negative result is retained because it limits the generality of the headline aggregate comparison.

Segment-Level Allocation Patterns. The strategy appendix reports segment capture under Context + Adaptive for baseline, price war, and supply shock. The numbers are not market shares that sum to 100 across a row; they are the project’s segment-specific capture indicators for each provider.

The table shows the intended role tension. PremiumCloud is strongly exposed to enterprise demand and therefore also to SLA failure under shock. SpotBroker’s spot indicator rises to 51.49% during price war. Hyperscaler reaches 50.30% on the training indicator during supply shock. Because role priors are encoded in the environment, these patterns demonstrate consistency between the designed heterogeneity and realized behavior rather than spontaneous discovery from identical initial agents.

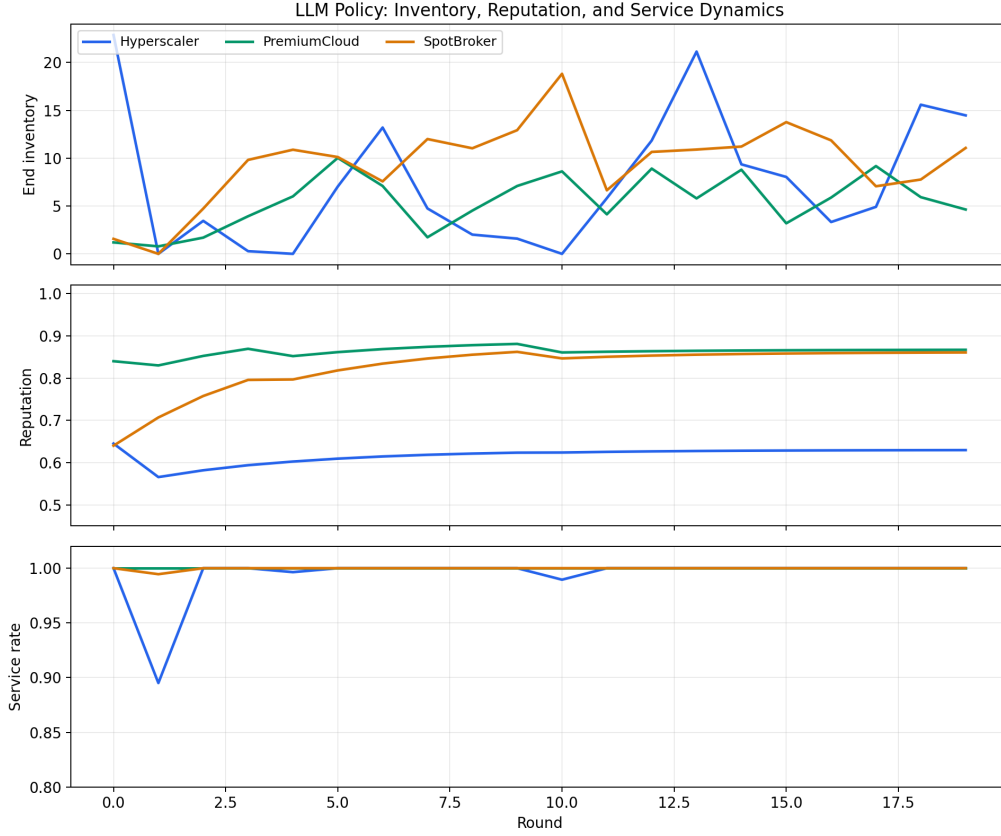


Figure 3: Provider state dynamics produced by the repository’s analysis pipeline.

Agent-Level Strategy States. Under Context + Adaptive in the supply shock, the final six-dimensional states differ markedly. Hyperscaler has risk tolerance 0.91, price aggressiveness 0.79, demand sensitivity 0.77, inventory caution 0.15, shock responsiveness 0.59, and competitor reactivity 0.63. PremiumCloud ends at 0.36, 0.48, 0.57, 0.70, 0.46, and 0.25. SpotBroker ends at 0.68, 0.82, 0.65, 0.83, 0.94, and 0.95.

These states match the qualitative roles encoded at initialization. Hyperscaler treats the shock as an expansion opportunity; PremiumCloud remains inventory-cautious and less competitor-reactive; SpotBroker is highly responsive to shocks and rivals. The same scenario yields approximate average prices of 5.02, 6.77, and 5.77 for Hyperscaler, PremiumCloud, and SpotBroker, with final reputation around 82%, 69%, and 92%, respectively.

The interpretation should remain bounded by the simulator. PremiumCloud’s lower final reputation is not a statement about real premium cloud providers; it follows from the combination of enterprise-heavy demand, the simulated shortage process, and the project’s reputation update rule.

5 Discussion

5.1 Failure Cases and Limitations

The project explicitly documents four cases where the scenario label is stronger than the observed effect. First, price-war average prices fall by less than 0.03 because price floors, the Risk Gate, and bounded states prevent sustained undercutting. Second, high volatility does not strongly separate policies over only 12 rounds. Third, supply-shock profit does not fall because emergency procurement and capacity-expansion costs are absent. Fourth, no reputation still leaves substantial provider heterogeneity through brand, role, capacity, cost, and floor price.

These are not incidental caveats; they delimit what the simulator can support. The experiment also uses one random seed, synthetic demand, and no calibration to real GPU transactions. It omits SKU heterogeneity, geography, electricity costs, long-term contracts, migration costs, and capacity lead times. Prices and profits therefore have meaning only inside the simulator.

The implementation includes the market environment, provider-agent pipeline, experiment runners, and CSV result outputs. All quantitative comparisons reported below correspond to the recorded experiment outputs described in the evaluation setup.

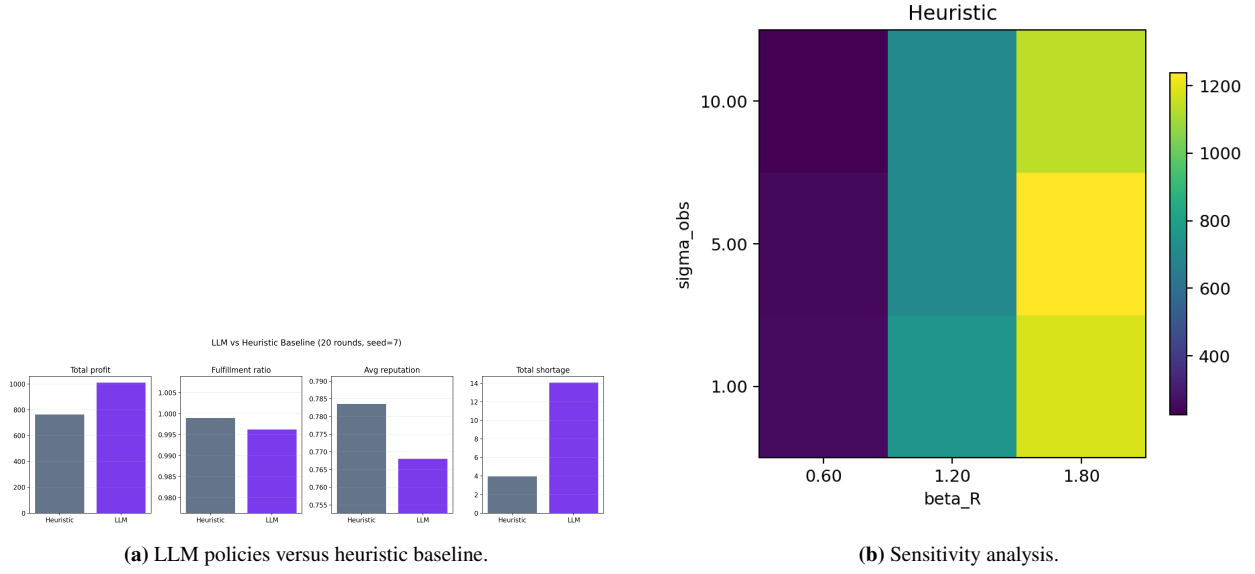


Figure 4: Comparative and sensitivity evidence generated in the PJ-AG4 repository.

Ablation Results. The two mechanism ablations help separate policy behavior from the support mechanisms implemented by the environment. In the no-reputation scenario, profits for Heuristic, Direct, Context, Adaptive, and Context + Adaptive are 3062, 3183, 3372, 3533, and 3693, respectively. In the no-transfer scenario the corresponding profits are 3312, 3435, 3356, 3559, and 3591. The combined adaptive policy therefore remains strong after either mechanism is removed, although the service variables change.

No reputation does not create homogeneous providers: brand strength, role, cost, capacity, and price floors remain different. Likewise, no transfer does not remove all shortage recovery because customer reallocation remains active. The ablations measure marginal contributions of two mechanisms under the rest of the simulator, not a complete removal of provider heterogeneity or market flexibility.

Scenario Profit Decomposition. The strategy appendix reports cumulative provider profit under Context + Adaptive for all six scenarios. Hyperscaler records 1523, 1446, 1604, 1389, 1421, and 1321 from baseline through no-transfer; PremiumCloud records 1038, 1045, 1301, 1180, 1043, and 1082; SpotBroker records 1262, 1010, 1422, 1176, 1230, and 1188. Hyperscaler is therefore the highest-profit provider in each reported scenario under this policy.

The result is consistent with the encoded scale advantage and should not be mistaken for an empirical ranking of real providers. More informative is the way the other two roles respond. SpotBroker is relatively strong in the supply shock while maintaining 99.07% fulfillment and zero defaults, which is consistent with its flexible spot-oriented design. PremiumCloud charges the highest average price in that shock but experiences greater service pressure because of its enterprise-heavy allocation.

Relation to Agent-Based Simulation. The architecture follows a standard agent-based separation between environment rules and heterogeneous decision makers, but replaces some fixed behavioral rules with an LLM-mediated policy. Generative-agent work has shown that language models can maintain behavior over repeated interactions when memory and context are structured rather than passed as an undifferentiated transcript.[1] The present project applies a narrower idea: context is compressed, action space is bounded, and economic settlement remains deterministic.

This design choice is important for evaluation. If the LLM were allowed to rewrite market rules, invent transactions, or bypass capacity limits, profit comparisons would be difficult to interpret. Here, the model can affect forecast, price, supply, or strategy state depending on mode, but the environment remains the final authority on feasible actions and pay-offs. The experiment therefore evaluates decision interfaces inside a fixed simulator rather than open-ended economic reasoning.

Scope of Inference and Further Validation. The current evidence is sufficient to compare policies inside one synthetic market, but not to claim that the same ranking would hold in a real GPU market. A stronger study would first repeat the experiment over many demand seeds and, ideally, repeated model calls. It would then vary horizon length, provider priors, customer-segment weights, capacity limits, and penalty coefficients to test whether the adaptive advantage survives reasonable perturbations.

A second step would calibrate the environment to observable market quantities: rental-price ranges, contract structure, utilization, supply lead times, electricity and depreciation costs, and empirical demand shocks. The provider

archetypes could then be interpreted as calibrated stylizations rather than hand-designed roles. A third step would compare multiple language models and a non-LLM adaptive controller to separate the value of the bounded interface from the value of the language model itself.

These extensions do not invalidate the present result. They mark the boundary between the current mechanism demonstration and a calibrated empirical market model.

6 Conclusion

The experiment suggests that more context is not automatically better for direct LLM control. In this simulator, the stronger design is to let the model update a bounded strategic state while deterministic code handles action construction, risk limits, and settlement. Context + Adaptive produces the highest aggregate profit and the lowest aggregate mean final backlog in the reported run, with its clearest advantage appearing under the supply shock. These findings are evidence about one synthetic mechanism and one experimental seed. A stronger claim would require multi-seed runs, additional models, longer horizons, and calibration against real market data.

References

- [1] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative Agents: Interactive Simulacra of Human Behavior," in *Proceedings of UIST*, 2023.
- [2] L. Tesfatsion and K. L. Judd, eds., *Handbook of Computational Economics, Vol. 2: Agent-Based Computational Economics*. Elsevier, 2006.